Marevlo Research / AI Foundations • Module 02 – Quantization • Topic 1

Uniform & Affine Quantization, Worked Out by Hand

How a floating-point weight becomes a small integer — and exactly how much error that costs

What this document does. It takes one small, concrete weight vector — eight numbers from a transformer FFN row — and *crushes* it into INT8 and INT4 by hand. We derive the affine map from first principles (scale s, zero-point z, $q = \text{round}(x/s) + z$, dequant $\hat{x} = s(q - z)$), predict the rounding error with the $s/\sqrt{12}$ model, and then verify every figure against a reference script (Appendix A, `m02_t01_uniform_quantization_engine.py`). By the end you can quantize a tensor on paper, choose symmetric vs. asymmetric for a reason, and explain why *group-wise* granularity (g=128) is the production default.

The one idea. Quantization is just choosing a *ruler*. A b-bit integer gives you 2^b tick marks; the scale s is the spacing between ticks. Every weight snaps to its nearest tick, and the leftover — at most half a tick — is the quantization error. Fewer bits means a coarser ruler means bigger error. Everything else (symmetry, granularity, the fancy methods in later topics) is about spending those scarce ticks where the data actually is.

Concepts covered, in order

the affine map $x \leftrightarrow q$; deriving scale s and zero-point z; symmetric vs. asymmetric (affine) schemes; round-to-nearest and the quantization-error model $\sigma_e = s/\sqrt{12}$; a full INT8 *and* INT4 worked example; group-wise (g=128) granularity and its storage overhead; the compression / bandwidth payoff.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

Contents

1 The object under study 2

2 Step 1 — The affine map 2

3 Step 2 — Symmetric quantization: derive s, set $z = 0$ 3

3.1 The full INT4 symmetric result 3

4 Step 3 — Asymmetric (affine) quantization: derive s and z 3

5 Step 4 — How big is the error? The $s/\sqrt{12}$ model 4

6 Step 5 — Granularity: per-tensor vs. group-wise 5

7 Step 6 — The payoff: compression is bandwidth 6

8 The argument, at a glance 6

8.1 Equation cheat-sheet 6

Appendix A — Reference implementation **6**

1 The object under study

We quantize one short weight vector. Real layers have millions of these; the arithmetic is identical, just repeated. Working eight numbers by hand keeps every quantity checkable.

Quantity	Symbol	Value here
Weight vector	W	$[-0.92, -0.10, 0.05, 0.21, -0.34, 0.62, 0.00, 0.41]$
Count	n	8
Minimum	$x_{\min}$	-0.92
Maximum	$x_{\max}$	0.62
Absolute max	$\max \lvert x \rvert$	0.92
Target widths	b	8 bits (INT8), 4 bits (INT4)

Intuition

Think of a thermometer. The mercury can sit anywhere (continuous, like a float), but the printed marks only let you *read* whole degrees (discrete, like an integer). Quantization is choosing how far apart the marks are and then reading each weight to the nearest mark. A finer thermometer (more marks = more bits) reads more precisely but costs more to print (more storage, more bytes to move). The whole game is reading accurately with as few marks as possible.

2 Step 1 — The affine map

Every uniform quantizer is the same straight line: a real value x maps to an integer code q, and back again, through a scale s (the spacing of the ruler) and an integer offset z (which code represents zero):

The affine quantize / dequantize pair

$$q \;=\; \mathrm{clip}\Big(\mathrm{round}\big(\tfrac{x}{s}\big) + z,\; q_{\min},\; q_{\max}\Big), \qquad \hat{x} \;=\; s\,(q - z).$$

Two numbers fully specify the scheme: s (how big one integer step is, in the original units) and z (the integer that lands exactly on $x{=}0$). Choosing them is the entire design decision, and the two canonical choices — *symmetric* ($z{=}0$) and *asymmetric* ($z \neq 0$) — are Steps 2 and 3.

Why this is the only sensible map

We want (i) a *uniform* grid of representable values so the kernel is a single multiply, and (ii) the grid to cover the data range $[x_{\min}, x_{\max}]$ using all 2^b codes. A uniform grid through the origin is $\hat{x} = s(q - z)$ — a line with slope s and intercept $-sz$. Inverting gives $q = x/s + z$; rounding to the nearest integer is the closest grid point; clipping keeps q inside the b-bit code range. There is nothing else to choose.

3 Step 2 — Symmetric quantization: derive s, set $z = 0$

Weights in trained transformers are roughly zero-centered, so the simplest choice forces the grid to be symmetric about zero: $z = 0$. The code range for b bits is $[-(2^{b-1}), 2^{b-1} - 1]$ — e.g. INT4 uses $[-8, 7]$, INT8 uses $[-128, 127]$.

Scale from the absolute max

To make the most extreme weight land on the largest *positive* code $q_{\max} = 2^{b-1} - 1$, set the spacing so that $\max|x|$ maps to $q_{\max}$:

$$s = \frac{\max|x|}{2^{b-1} - 1}.$$

For our vector, $\max|x| = 0.92$. At INT4, $q_{\max} = 2^3 - 1 = 7$:

$$s_4 = \frac{0.92}{7} = 0.131429.$$

At INT8, $q_{\max} = 127$: $s_8 = 0.92/127 = 0.007244$. ✓

Mini-example — quantizing one weight by hand

Take $x = 0.41$ at INT4 symmetric, $s_4 = 0.131429$:

$$\frac{x}{s_4} = \frac{0.41}{0.131429} = 3.12 \xrightarrow{\text{round}} q = 3, \qquad \hat{x} = s_4 \cdot 3 = 0.3943.$$

The error is $0.41 - 0.3943 = +0.0157$ — about an eighth of one tick. The outlier $x = 0.62$ goes to $q = \text{round}(0.62/0.131429) = \text{round}(4.72) = 5$, $\hat{x} = 0.6571$, error -0.0371. Every weight snaps to its nearest multiple of s_4.

3.1 The full INT4 symmetric result

x	x/s_4	q	$\hat{x} = s_4 q$	error $x - \hat{x}$
−0.92	−7.00	−7	−0.9200	0.0000
−0.10	−0.76	−1	−0.1314	0.0314
0.05	0.38	0	0.0000	0.0500
0.21	1.60	2	0.2629	−0.0529
−0.34	−2.59	−3	−0.3943	0.0543
0.62	4.72	5	0.6571	−0.0371
0.00	0.00	0	0.0000	0.0000
0.41	3.12	3	0.3943	0.0157

The codes are $q = [-7, -1, 0, 2, -3, 5, 0, 3]$, RMSE $= 0.036836$, $\max|e| = 0.054286$.

4 Step 3 — Asymmetric (affine) quantization: derive s and z

When the data is *not* centered at zero (post-ReLU/SiLU activations are the classic case), forcing a symmetric grid wastes half the codes on a sign the data never uses. The affine scheme spends the full unsigned range $[0,\ 2^b - 1]$ on the actual interval $[x_{\min}, x_{\max}]$.

Scale and zero-point

Spread $2^b - 1$ steps across the data span:

$$s = \frac{x_{\max} - x_{\min}}{2^b - 1}.$$

The zero-point is the code that maps to $x = 0$. Setting $\hat{x} = s(q - z) = 0$ at the code for $x_{\min}$ gives $z = \text{round}(-x_{\min}/s)$. For INT4 ($2^b - 1 = 15$):

$$s_4 = \frac{0.62 - (-0.92)}{15} = \frac{1.54}{15} = 0.102667, \qquad z = \text{round}\Big(\frac{0.92}{0.102667}\Big) = \text{round}(8.96) = 9.$$

So integer code 9 represents the true zero, and $x_{\min} = -0.92 \mapsto q = 0$, $x_{\max} = 0.62 \mapsto q = 15$. ✓

Mini-example — the same weight, affine INT4

$x = 0.41$, $s_4 = 0.102667$, $z = 9$:

$$q = \text{round}\big(\tfrac{0.41}{0.102667}\big) + 9 = \text{round}(3.99) + 9 = 13, \qquad \hat{x} = s_4(13 - 9) = 0.4107,$$

error -0.0007 — far smaller than the symmetric case, because the affine grid did not waste codes below -0.92. Full INT4 affine: $q = [0, 8, 9, 11, 6, 15, 9, 13]$, RMSE $= 0.021170$.

Symmetric vs. asymmetric — the real tradeoff

Asymmetric uses the codes more efficiently when the data is lopsided (here it cut INT4 RMSE from 0.0368 to 0.0212, a 43% reduction). But it costs a *subtract-the-zero-point* in every matmul — $\hat{x} = s(q - z)$ does not factor out of a dot product the way $\hat{x} = sq$ does. Production rule of thumb: **symmetric for weights** (already zero-centered, keep the kernel cheap), **asymmetric / dynamic per-token for activations** (lopsided after ReLU/SiLU).

5 Step 4 — How big is the error? The $s/\sqrt{12}$ model

Round-to-nearest snaps each x to its nearest grid point, so the residual $e = x - \hat{x}$ lies in $[-s/2, +s/2]$. If x falls anywhere in a cell with no preference, e is approximately *uniform* on that interval.

Standard deviation of a uniform error

For $e \sim \text{Uniform}(-\frac{s}{2}, \frac{s}{2})$, the variance is

$$\sigma_e^2 = \frac{1}{s}\int_{-s/2}^{s/2} e^2\, de = \frac{s^2}{12} \quad \Longrightarrow \quad \boxed{\sigma_e = \frac{s}{\sqrt{12}}}.$$

The error scales *linearly* with the tick spacing s. Halve s (one more bit) and you halve the expected error.

Quantization-noise rule of thumb

$$\sigma_e \approx \frac{s}{\sqrt{12}} = \frac{s}{3.464}.$$

How well does it predict our measured RMSE? Compare model vs. reality:

Scheme	scale s	$s/\sqrt{12}$ (model)	measured RMSE
INT8 symmetric	0.007244	0.002091	0.001588
INT4 symmetric	0.131429	0.037940	0.036836
INT4 asymmetric	0.102667	0.029637	0.021170

The model nails the symmetric INT4 RMSE (0.0379 vs. 0.0368). It slightly over-predicts on small samples (only 8 weights) and where a couple of weights sit near grid points, but the *scaling law* — error proportional to s — is exactly right.

Watch out

The $s/\sqrt{12}$ model assumes errors are uncorrelated and zero-mean. The dangerous failure mode is *bias*: if a quantizer systematically rounds one direction (e.g. clamping all outliers down), the per-layer errors stop cancelling and instead *compound* across depth. Avoiding bias — not minimizing per-weight RMSE — is most of the quantization game. This is exactly the hole that GPTQ and AWQ (Topics 2–3) fill.

6 Step 5 — Granularity: per-tensor vs. group-wise

So far one scale covered all eight weights (*per-tensor*). But the outlier -0.92 inflated s for everyone, so the small weights near zero got a coarse ruler. **Group-wise** quantization gives each block of g consecutive weights its own scale, so a local block free of outliers gets a finer ruler.

Group-wise INT4 with $g = 4$

Split the vector into two groups of 4 and quantize each symmetrically with its own s:

$$\begin{aligned} &\text{group 0: } [-0.92, -0.10, 0.05, 0.21] \quad \max|x| = 0.92 \quad s_0 = 0.92/7 = 0.131429, \\ &\text{group 1: } [-0.34, 0.62, 0.00, 0.41] \quad \max|x| = 0.62 \quad s_1 = 0.62/7 = 0.088571. \end{aligned}$$

Group 1 no longer sees the big -0.92, so its ruler is 33% finer (0.0886 vs. 0.1314). Group-wise RMSE = 0.030752 vs. per-tensor INT4 0.036836 — a 16.5% error reduction at INT4, for free except for storing one extra scale. ✓

Intuition

One scale for the whole tensor is like one shoe size for a whole family — the giant's foot sets the size and the toddler is swimming in it. Group-wise is per-person sizing: each small neighbourhood of weights gets a scale matched to its own magnitude, so no group is forced to waste resolution covering a far-away outlier it does not contain.

Why $g = 128$ is the production sweet spot

Finer groups always lower error, but each group stores an extra (FP16) scale. With $g = 128$, the overhead is one 16-bit scale per 128 weights = 16/128 = 0.125 bits/weight = 12.5% on top of INT4's 4 bits. Going to $g = 32$ quadruples that overhead for diminishing accuracy gain. AWQ and GPTQ report <0.5% accuracy loss at **W4** g=128, versus 2–3% for plain round-to-nearest at the same width — which is why g=128 is the default everywhere.

7 Step 6 — The payoff: compression is bandwidth

The point of all this is fewer bytes. In the memory-bound decode regime (Module 01), fewer bytes per weight is *directly* fewer bytes across HBM per token — i.e. proportionally faster.

Precision	bytes/weight	compression vs. FP16	70B model size
FP16	2	1.0×	140 GB
INT8	1	2.0×	70 GB
INT4	0.5	4.0×	35 GB
INT4 g=128	0.5625	3.6×	39.4 GB

Memory and speed are the same axis

Cutting FP16 → INT4 is 4× less HBM traffic per decode step. In the batch-1 memory-bound regime that is a 4× *speedup*, not merely a memory saving — the bytes you do not move are time you do not spend. This is why INT4 is the default starting point for serving: nearly free speed, with sub-1% accuracy cost when done with a method-aware quantizer.

8 The argument, at a glance

#	Step	Result
1	Affine map	$q = \text{round}(x/s) + z, \quad \hat{x} = s(q - z)$
2	Symmetric s	$s = \max\|x\|/(2^{b-1}-1)$; INT4 s_4=0.1314
3	Asymmetric s, z	$s = (x_{\max}-x_{\min})/(2^b-1)$, $z = \text{round}(-x_{\min}/s)$=9
4	Error model	$\sigma_e = s/\sqrt{12}$; INT4-sym 0.0379 ≈ measured 0.0368
5	Granularity	g=4 cuts INT4 RMSE 0.0368 → 0.0308 (−16.5%)
6	Payoff	INT4 = 4× smaller = 4× faster decode

8.1 Equation cheat-sheet

Quantize	$q = \text{clip}(\text{round}(x/s) + z,\ q_{\min}, q_{\max})$
Dequantize	$\hat{x} = s\,(q - z)$
Symmetric scale	$s = \max\|x\| \,/\, (2^{b-1} - 1), \quad z = 0$
Affine scale	$s = (x_{\max} - x_{\min}) \,/\, (2^b - 1)$
Zero-point	$z = \text{round}(-x_{\min}/s)$
Error model	$\sigma_e = s/\sqrt{12}$
Group overhead	(scale bits)/g extra bits/weight

Appendix A — Reference implementation

Every number above is produced by `m02_t01_uniform_quantization_engine.py`. Run it to reproduce all figures; change W or `bits` to quantize your own vectors.

```
"""
Reference implementation for: Module 02 - Topic 1
"Uniform / Affine Quantization - crushing weights into integers."

Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
```

```
so the arithmetic in the document is exact and self-consistent.

We quantize one small FFN-style weight vector to INT8 and INT4 using both
symmetric and asymmetric (affine) schemes, then measure error.
"""

import numpy as np

np.set_printoptions(precision=4, suppress=True)

# ---------------------------------------------------------------------
# 0. The toy weight vector (8 weights from a transformer FFN row).
#    Mostly small, zero-ish centered, with one mild outlier (0.62).
# ---------------------------------------------------------------------
W = np.array([-0.92, -0.10, 0.05, 0.21, -0.34, 0.62, 0.00, 0.41])
print("== 0. The weight vector ==")
print("W      =", W)
print(f"min={W.min():.4f}  max={W.max():.4f}  absmax={np.abs(W).max():.4f}  n={W.size}")


# ---------------------------------------------------------------------
# Quantizer primitives.
# ---------------------------------------------------------------------
def symmetric_quant(W, bits):
    """Symmetric: q = round(W/s), s = absmax/(2^(b-1)-1).  Range [-(2^(b-1)), 2^(b-1)-1]."""
    qmax = 2 ** (bits - 1) - 1          # e.g. INT4 -> 7,  INT8 -> 127
    qmin = -(2 ** (bits - 1))           # e.g. INT4 -> -8, INT8 -> -128
    s = np.abs(W).max() / qmax
    q = np.clip(np.round(W / s), qmin, qmax)
    W_hat = s * q
    return s, 0, q, W_hat, qmin, qmax


def asymmetric_quant(W, bits):
    """Affine: s=(max-min)/(2^b-1), z=round(-min/s), q=clip(round(W/s)+z,0,2^b-1)."""
    qmax = 2 ** bits - 1                # INT4 -> 15, INT8 -> 255
    s = (W.max() - W.min()) / qmax
    z = int(np.round(-W.min() / s))
    q = np.clip(np.round(W / s) + z, 0, qmax)
    W_hat = s * (q - z)
    return s, z, q, W_hat, 0, qmax


def report(name, s, z, q, W_hat, W):
    err = W - W_hat
    rmse = np.sqrt(np.mean(err ** 2))
    maxe = np.abs(err).max()
    theo = s / np.sqrt(12)              # uniform-rounding error model: std = s/sqrt(12)
    print(f"\n== {name} ==")
    print(f"scale s   = {s:.6f}")
    print(f"zero-pt z = {z}")
    print(f"q (int)   = {q.astype(int)}")
    print(f"W_hat     = {W_hat}")
    print(f"error     = {err}")
    print(f"RMSE      = {rmse:.6f}   max|err| = {maxe:.6f}")
    print(f"theory s/sqrt(12) = {theo:.6f}  (expected error std)")
    return rmse, maxe, theo


# ---------------------------------------------------------------------
# 1. INT8 symmetric and asymmetric.
# ---------------------------------------------------------------------
s8s, z8s, q8s, W8s, *_ = symmetric_quant(W, 8)
```

```python
r8s = report("1a. INT8 symmetric (per-tensor)", s8s, z8s, q8s, W8s, W)

s8a, z8a, q8a, W8a, *_ = asymmetric_quant(W, 8)
r8a = report("1b. INT8 asymmetric (affine, per-tensor)", s8a, z8a, q8a, W8a, W)

# ------------------------------------------------------------------------
# 2. INT4 symmetric and asymmetric (much coarser).
# ------------------------------------------------------------------------
s4s, z4s, q4s, W4s, *_ = symmetric_quant(W, 4)
r4s = report("2a. INT4 symmetric (per-tensor)", s4s, z4s, q4s, W4s, W)

s4a, z4a, q4a, W4a, *_ = asymmetric_quant(W, 4)
r4a = report("2b. INT4 asymmetric (affine, per-tensor)", s4a, z4a, q4a, W4a, W)

# ------------------------------------------------------------------------
# 3. Group-wise INT4 (g=4): split the 8 weights into two groups of 4,
#    each with its OWN symmetric scale. Finer granularity -> lower error.
# ------------------------------------------------------------------------
print("\n== 3. Group-wise INT4 symmetric, group size g=4 ==")
g = 4
W_hat_grp = np.empty_like(W)
for gi in range(0, W.size, g):
    blk = W[gi:gi + g]
    s, _, q, blk_hat, *_ = symmetric_quant(blk, 4)
    W_hat_grp[gi:gi + g] = blk_hat
    print(f"  group {gi//g}: W={blk}  s={s:.6f}  q={q.astype(int)}  W_hat={blk_hat}")
err_grp = W - W_hat_grp
rmse_grp = np.sqrt(np.mean(err_grp ** 2))
print(f"  group-wise INT4 RMSE = {rmse_grp:.6f}   (vs per-tensor INT4 sym {r4s[0]:.6f})")

# ------------------------------------------------------------------------
# 4. Summary table of RMSE across schemes.
# ------------------------------------------------------------------------
print("\n== 4. RMSE summary (lower is better) ==")
print(f"  INT8 symmetric            RMSE = {r8s[0]:.6f}")
print(f"  INT8 asymmetric           RMSE = {r8a[0]:.6f}")
print(f"  INT4 symmetric per-tensor RMSE = {r4s[0]:.6f}")
print(f"  INT4 asymmetric per-tensor RMSE= {r4a[0]:.6f}")
print(f"  INT4 symmetric g=4        RMSE = {rmse_grp:.6f}")
print(f"\n  INT4-sym error is {r4s[0]/r8s[0]:.1f}x the INT8-sym error")
print(f"  group-wise cuts INT4 error by {(1-rmse_grp/r4s[0])*100:.1f}%")

# ------------------------------------------------------------------------
# 5. Memory / compression.
# ------------------------------------------------------------------------
print("\n== 5. Compression vs FP16 ==")
for bits in (8, 4):
    print(f"  INT{bits}: {16/bits:.1f}x smaller   70B -> {70*bits/8:.1f} GB")
print(f"  group-wise INT4 g=128 scale overhead = {16/128*100:.2f}% extra bits/weight")
```

— END OF DOCUMENT —